

Manual de Construção — Painel de ROs

Como o pipeline de acompanhamento de ROs (Registros de Oportunidade) e a integração com o Radar foram construídos

o. Uma distinção importante antes de começar

O "Painel de ROs" que aparece dentro do NTSEC Radar é composto de **duas coisas separadas**, construídas em momentos e por processos diferentes:

PEÇA	O QUE É	ONDE VIVE
Gestão de ROs	Um aplicativo Streamlit (Python) completo e independente, com pipeline de dados, alertas por e-mail e um painel de monitoramento. Já existia antes de ser embutido no Radar.	Pasta própria <code>Gestao_ROs_Bitrix</code> , fora do projeto do Radar.
O encaixe no Radar	Um mecanismo pequeno, dentro do próprio Radar, que sobe o Streamlit acima como um processo em segundo plano e o exhibe dentro de uma moldura (<code>iframe</code>), para não precisar de uma janela separada.	Dentro do projeto do Radar, em <code>app/server.js</code> .

Este manual cobre as duas coisas: primeiro como o aplicativo Gestão de ROs foi montado (para quem for adaptar ou recriar o pipeline), depois como o Radar o embute (para quem for integrar outro painel do mesmo jeito).

Parte A — O aplicativo Gestão de ROs

O que ele resolve

Uma **Oportunidade de Renovação (RO)** tem uma data de vencimento. A empresa precisa saber, com antecedência, quais estão perto de vencer, avisar o vendedor (AM) responsável, e depois consolidar as decisões de todos num relatório único. O CRM (Bitrix) tem os dados brutos, mas não organiza isso por vendedor nem dispara aviso sozinho — é isso que este pipeline faz por cima do export do CRM.

Fase 1 — A fonte de dados

Tudo começa com um arquivo exportado manualmente do Bitrix CRM (formato `report (XX).xls`), colocado numa pasta de entrada. Cada linha do export é um negócio, com colunas de até três ROs por negócio (1º RO, 2º RO, 3º RO), cada bloco com sua própria data de vencimento.

PROMPT PARA COLAR NA IA

Preciso de um script Python que leia um arquivo .xls exportado de um CRM (formato Bitrix), onde cada linha é um "negocio" e há até 3 blocos de colunas repetidos lado a lado (1o RO, 2o RO, 3o RO), cada bloco com: aprovado (sim/nao), identificador do RO, data de vencimento.

O script deve:

1. Ler o .xls mais recente de uma pasta configurável via variável de ambiente.
2. Para cada negocio, calcular para cada bloco de RO preenchido: se está vencido (data de vencimento no passado) e quantos dias faltam ou passaram.
3. Separar os negocios por VENDEDOR RESPONSÁVEL (uma coluna do export) e gravar uma planilha .xlsx por vendedor, cada uma com duas abas: "Dados" (tudo) e "Vencimento" (so os negocios com pelo menos um RO vencendo nos proximos 30 dias, e que o vendedor ainda não tenha decidido "Não renovar").
4. IMPORTANTE: se a planilha do vendedor já existir de uma execução anterior e tiver uma coluna "Decisao" preenchida a mão, o script deve preservar essas decisoes ao atualizar (merge pelo identificador do negocio), nunca sobrescrever o que a pessoa já preencheu.

VERIFICAÇÃO

Rode com um export de teste (pode ser uma planilha fake com 2-3 linhas). Confira que saiu uma planilha por vendedor, com a aba "Vencimento" só trazendo quem realmente está no prazo de risco.

Fase 2 — Alertas automáticos por e-mail

Depois que as planilhas por vendedor existem, um segundo script varre cada uma e dispara um e-mail para quem tem RO vencendo.

PROMPT PARA COLAR NA IA

Crie um segundo script Python que:

1. Le a aba "Dados" de cada planilha de vendedor gerada na fase anterior.
2. Para cada negocio com algum RO vencido ou vencendo nos proximos 30 dias (e sem decisao "Nao renovar" ja marcada), monta um e-mail em HTML listando os ROs em risco.
3. Envia usando OUTLOOK COM local (nao SMTP direto) - assim usa a conta ja logada na maquina, sem guardar senha em lugar nenhum.
4. Tenha uma variavel de ambiente tipo ALERTAS_APENAS_EXIBIR: se ligada, o e-mail so ABRE na tela para conferencia manual antes de enviar, nunca envia sozinho automaticamente.

O mapeamento de qual vendedor recebe em qual e-mail deve vir de um arquivo de configuracao (.env ou json), nunca escrito fixo no codigo.

VERIFICAÇÃO

Ligue primeiro o modo "apenas exibir" e confirme que o e-mail mostrado está correto antes de qualquer teste de envio de verdade.

Fase 3 — Consolidar tudo num relatório único

PROMPT PARA COLAR NA IA

Crie um terceiro script que junta as planilhas de TODOS os vendedores (com as decisoes ja preenchidas por eles) num unico arquivo consolidado .xlsx, formatado para leitura gerencial: cabecalho colorido, tabela nomeada (Excel Table, nao so celulas soltas), largura de coluna ajustada ao conteudo. Use a biblioteca openpyxl para ler e escrever formatacao (Font, PatternFill, Border, Table).

VERIFICAÇÃO

Abra o consolidado final e confirme que todo vendedor que tinha planilha individual aparece nele, com as decisões preservadas.

O painel Streamlit (a interface)

Por cima dos três scripts, um app Streamlit dá uma tela única com **quatro seções**, trocadas por um menu de rádio na barra lateral — não por abas separadas.

PROMPT PARA COLAR NA IA

Crie um app.py em Streamlit com navegacao por st.sidebar.radio (nao st.tabs) entre 4 telas:

1. "Painel de Monitoramento (ROs)": le o consolidado mais recente e mostra cada negocio como um CARTAO HTML sanitizado (nao uma tabela crua) - com o nome do negocio, vendedor, fabricante, quantos dias faltam para o vencimento e a decisao do AM em destaque. Use o MESMO SISTEMA VISUAL do painel de Boletins (mesmas cores, mesmo estilo de card), para as duas telas parecerem parte do mesmo produto. Ofereca tambem um modo de tabela (st.dataframe) como alternativa para quem prefere ver tudo compacto, e um botao de download da visualizacao atual em Excel. Tenha um filtro de "janela de vigilancia" na barra lateral: radio para escolher entre "so vencidos", "so futuros" (com slider de quantos dias a frente) ou "vencidos + futuros". Pagine a tabela (25/50/100/todos por pagina).
2. "Configuracoes de Fabricantes": uma tela que le e escreve um config_fabricantes.json com regra por fabricante (ex: renovacao automatica sim/nao, quantos dias de prazo de extensao) - pessoa de negocio edita sem tocar em codigo.
3. "Executar Scripts": le um scripts.json listando os scripts do pipeline (titulo, descricao, caminho do arquivo, pasta de execucao, arquivo .env a usar) - mostra um seletor, um botao "Ver codigo fonte" (st.code) e um botao "Rodar", que executa via subprocess.run com captura de saida e mostra o resultado na tela.
4. "Manual de Uso": renderiza um arquivo Markdown de ajuda direto na tela (assim o manual nunca fica desatualizado em relacao ao app - e o mesmo arquivo usado dentro e fora).

Carregue os dados do consolidado com cache do Streamlit (`st.cache_data`) para não reler o Excel a cada interação - e tenha um botão "Recarregar" que limpa esse cache manualmente.

VERIFICAÇÃO

Rode `streamlit run app.py` e navegue pelas quatro seções. O filtro de janela de vigilância muda a lista? O botão Recarregar realmente relê o arquivo do disco (edite algo nele por fora e confirme)?

ATENÇÃO

Variáveis de ambiente (caminhos de pasta, e-mails, mapeamento de vendedor) devem viver **só** no arquivo `.env`, que precisa estar no `.gitignore` se este projeto for versionado, porque ele carrega nome de pessoa e link de compartilhamento pessoal — informação que não deve ir para nenhum repositório nem para qualquer documento distribuído.

Parte B — Como o Radar embute este painel

Esta parte descreve exatamente o que existe hoje dentro do NTSEC Radar. Não é uma proposta teórica.

Passo único — Subir o Streamlit como processo filho e mostrar num iframe

PROMPT PARA COLAR NA IA

No `server.js` do meu app Node, quero embutir um painel Streamlit já existente, que mora numa pasta separada com seu próprio `app.py`. Adicione:

1. Uma função que resolve o caminho do Python de verdade (não o atalho de 0 byte que a Microsoft Store deixa no PATH - descarte qualquer `python.exe` dentro de uma pasta "WindowsApps" ou com tamanho de arquivo zero).
2. Uma função `"painelNoAr()"` que tenta uma requisição HTTP rápida (timeout curto, tipo 1.5s) para `http://localhost:8501/` e devolve se respondeu ou não - serve tanto para não subir um processo duplicado quanto para detectar se alguém já abriu o painel por fora (por um atalho `.bat`, por exemplo).
3. Rota POST `/api/painel-subir`: se `painelNoAr()` já for verdadeiro, só devolve sucesso sem fazer nada. Senão, usa `child_process.spawn` para rodar `python -m streamlit run app.py --server.port 8501 --server.headless true --browser.gatherUsageStats false` com `cwd` apontando pra pasta do painel, guarda a saída (`stdout/stderr`) numa variável para debug, e fica tentando `painelNoAr()` a cada meio segundo por até 20 segundos antes de desistir e devolver erro com o log capturado.
4. Rota POST `/api/painel-parar`: mata o processo guardado, se existir.
5. Rota GET `/api/painel-status`: devolve se está no ar agora.
6. No HTML, um iframe apontando para `http://localhost:8501` que só aparece depois que o "subir" confirmar sucesso, com um botão separado "abrir em nova aba" (alguns recursos do Streamlit não funcionam bem dentro de iframe, então a pessoa pode preferir a aba separada).

VERIFICAÇÃO

Feche todo Streamlit rodando, clique "Abrir painel" no Radar e confirme que sobe sozinho. Depois, abra o mesmo painel MANUALMENTE por fora (fora do Radar) e clique "Abrir painel" de novo. O Radar deve reconhecer que já está no ar e não tentar subir um segundo processo.

Configuração de ambiente

Nem a pasta do painel nem o caminho do Python devem ficar fixos no código — resolvidos automaticamente ou lidos de `config/ambiente.json`, para o mesmo Radar funcionar em outra máquina sem editar código-fonte.

NTSEC Radar — Manual de Construção, Painel de ROs. A Parte A descreve o aplicativo Gestão de ROs tal como existe hoje; a Parte B descreve exatamente o mecanismo de encaixe implementado no Radar.